python

July 29, 2026

1 Crash Course in Python

`{note} This notebook is a standalone, self-paced primer for students who are new to Python or need a refresher. It assumes no prior programming experience and covers just enough Python and NumPy to comfortably read and write the code used in the rest of this course, especially Lecture 6 (Python Scipy Method), where we start solving linear programs computationally.`

Learning Objectives

By the end of this lecture, you will be able to: 1. Write and run Python code using variables, conditionals, loops, and functions. 2. Store and manipulate data using Python's core data structures — lists, tuples, dictionaries, and sets. 3. Create, index, and operate on NumPy arrays, including basic vector and matrix operations. 4. Translate a small numeric problem (e.g., computing costs, checking constraints) into working Python/NumPy code.

Prerequisites: None — this is a standalone primer.

Estimated time: 90–120 minutes.

> **Environment check**: Run the setup cell below before proceeding. This notebook requires `numpy` — available in standard Anaconda / Google Colab environments.

[ ]:
```
# --- Environment Setup ---
import numpy as np

print(f"numpy version : {np.__version__}")
print("Setup complete.")
```

1.1 Intuition

> **Why Python, and why now**: Every optimization technique in this course — from linear programs to metaheuristics — eventually needs to be *solved*, and by Lecture 6 you will be handing your formulations to `scipy.optimize.linprog`. Python is the common language for that: it reads close to plain English, has a massive scientific-computing ecosystem (NumPy, SciPy, pandas), and is free and open source. This notebook gives you just enough fluency to write and understand the code used in the rest of the course — it is not a full programming course.

1.2 Python Fundamentals

1.2.1 Variables and Data Types

Type	Example	Description
`int`	`n_routes = 4`	Whole numbers
`float`	`unit_cost = 12.5`	Decimal numbers
`str`	`mode = "rail"`	Text
`bool`	`is_feasible = True`	`True` / `False`

[ ]:
```python
n_routes = 4          # int
unit_cost = 12.5      # float
mode = "rail"         # str
is_feasible = True    # bool

print(n_routes, type(n_routes))
print(unit_cost, type(unit_cost))
print(mode, type(mode))
print(is_feasible, type(is_feasible))
```

1.2.2 Control Flow

`if` / `elif` / `else` branches on a condition; `for` loops step through a sequence; `while` loops repeat until a condition becomes false.

[ ]:
```python
route_cost = 850

if route_cost < 500:
    label = "cheap"
elif route_cost < 1000:
    label = "moderate"
else:
    label = "expensive"

print(f"Route cost {route_cost} is classified as '{label}'")
```

[ ]:
```python
route_costs = [420, 850, 1500, 300, 990]

for cost in route_costs:
    if cost < 500:
        label = "cheap"
    elif cost < 1000:
        label = "moderate"
    else:
        label = "expensive"
    print(f"{cost:>5} -> {label}")
```

[ ]:
```
budget = 2000
spent = 0
route_index = 0

while spent + route_costs[route_index] <= budget:
    spent += route_costs[route_index]
    route_index += 1

print(f"Routes funded: {route_index}, total spent: {spent}")
```

1.2.3 Functions

A function packages reusable logic behind a name. Use `def` to define it, parameters to pass data in, and `return` to send a result back out.

[ ]:
```
def total_cost(quantities, unit_costs):
    total = 0
    for q, c in zip(quantities, unit_costs):
        total += q * c
    return total

quantities = [10, 25, 15]
unit_costs = [12.5, 8.0, 20.0]

print("Total cost:", total_cost(quantities, unit_costs))
```

1.2.4 Data Structures

Structure	Ordered	Mutable	Duplicates	Example
`list`	Yes	Yes	Yes	`[420, 850, 1500]`
`tuple`	Yes	No	Yes	`(0, 0)`
`dict`	Yes (insertion)	Yes	Unique keys	`{"rail": 5.5}`
`set`	No	Yes	No	`{"rail", "road"}`

[ ]:
```
route_costs = [420, 850, 1500, 300, 990]
route_costs.append(675)

print(route_costs)
print("Cheapest route:", min(route_costs))
print("Number of routes:", len(route_costs))
```

[ ]:
```
origin_destination = ("Chennai", "Bengaluru")

print(origin_destination)
print("Origin:", origin_destination[0])
```

[ ]:
```python
cost_per_km = {"road": 8.0, "rail": 5.5, "air": 45.0}
cost_per_km["sea"] = 3.0

for transport_mode, cost in cost_per_km.items():
    print(f"{transport_mode:>5}: Rs. {cost}/km")
```

[ ]:
```python
modes_used_route_A = {"road", "rail"}
modes_used_route_B = {"rail", "air"}

print("Modes used in either route:", modes_used_route_A | modes_used_route_B)
print("Modes used in both routes:", modes_used_route_A & modes_used_route_B)
```

1.3 NumPy Essentials

Python lists are flexible but slow for numeric work. NumPy arrays store numbers in contiguous memory and support fast, vectorized operations — this is what `scipy.optimize.linprog` expects as input in Lecture 6.

1.3.1 Arrays and Creation

[ ]:
```python
costs = np.array([12.5, 8.0, 20.0])
zeros = np.zeros(4)
ones = np.ones((2, 3))
sequence = np.arange(0, 10, 2)
evenly_spaced = np.linspace(0, 1, 5)

print("costs          :", costs)
print("zeros          :", zeros)
print("ones (2x3)     :\n", ones)
print("arange(0,10,2) :", sequence)
print("linspace       :", evenly_spaced)
```

1.3.2 Indexing and Slicing

[ ]:
```python
distance_matrix = np.array([
    [0, 12, 8],
    [12, 0, 15],
    [8, 15, 0]
])

print("Full matrix:\n", distance_matrix)
print("Row 0 (distances from node 0):", distance_matrix[0])
print("Distance from node 1 to node 2:", distance_matrix[1, 2])
print("First two rows:\n", distance_matrix[:2])
print("Distances greater than 10:", distance_matrix[distance_matrix > 10])
```

1.3.3 Basic Linear Algebra

{tip} The expression `quantities @ unit_costs` below is exactly the pattern you will use to evaluate an LP objective `c @ x` in Lecture 6 (Python Scipy Method) - get comfortable with it now.

[ ]:
```
quantities = np.array([10, 25, 15])
unit_costs = np.array([12.5, 8.0, 20.0])

elementwise = quantities * unit_costs
total = quantities @ unit_costs        # equivalent to np.dot(quantities,␣
 ↪unit_costs)
row_sums = distance_matrix.sum(axis=1)

print("Element-wise cost per item :", elementwise)
print("Total cost (dot product)   :", total)
print("Row sums of distance matrix:", row_sums)
```

1.4 In-Class Exercises

Work through the following exercises using what you have learned so far. Solutions are shown below each problem — try writing your own code first before reading them.

1.4.1 Exercise 1 — Cheapest Supplier

A firm can source a component from three suppliers at the following unit costs ( ): Supplier A = 45, Supplier B = 38, Supplier C = 52. It needs 120 units.

Step 1 — Store the data in a list of names and a NumPy array of unit costs.

Step 2 — Compute the total cost from each supplier.

Step 3 — Identify the cheapest supplier using `np.argmin`.

[ ]:
```
suppliers = ["A", "B", "C"]
unit_costs = np.array([45, 38, 52])
demand = 120

total_costs = unit_costs * demand
cheapest_index = np.argmin(total_costs)

for supplier, cost in zip(suppliers, total_costs):
    print(f"Supplier {supplier}: Rs. {cost}")

print(f"\nCheapest supplier: {suppliers[cheapest_index]} (Rs.␣
 ↪{total_costs[cheapest_index]})")
```

1.4.2 Exercise 2 — Feasibility Check

Three routes have capacities of 100, 150, and 80 units. Write a function `is_feasible(quantities, capacities)` that returns `True` only if every route's shipment is within its capacity, then test it on

two candidate shipment plans.

[ ]:
```python
capacities = np.array([100, 150, 80])

def is_feasible(quantities, capacities):
    quantities = np.array(quantities)
    return bool(np.all(quantities <= capacities))

plan_1 = [90, 140, 75]
plan_2 = [90, 160, 75]

print("Plan 1 feasible:", is_feasible(plan_1, capacities))
print("Plan 2 feasible:", is_feasible(plan_2, capacities))
```

1.4.3 Exercise 3 — Mode Comparison

A shipment of 200 units can be sent by road, rail, or air, at the per-unit costs stored in the dictionary below. Build a NumPy array of total costs from the dictionary, then use `np.argmin` / `np.argmax` to report the cheapest and priciest modes.

[ ]:
```python
cost_per_unit = {"road": 8.0, "rail": 5.5, "air": 45.0}
shipment_size = 200

modes = list(cost_per_unit.keys())
total_costs = np.array([cost_per_unit[m] * shipment_size for m in modes])

cheapest = modes[np.argmin(total_costs)]
priciest = modes[np.argmax(total_costs)]

for m, c in zip(modes, total_costs):
    print(f"{m:>5}: Rs. {c:,.0f}")

print(f"\nCheapest mode : {cheapest}")
print(f"Priciest mode : {priciest}")
```

1.5 Take-Away Exercises

Complete the following exercises on your own before the next lecture. Fill in the code where indicated — do not just read the comments.

1.5.1 Exercise 1 — Total Network Cost

A transportation network has 5 routes with the unit costs and planned quantities given below. Using NumPy only (no explicit loops), compute (a) the cost per route, (b) the total network cost, and (c) which route is most expensive.

[17]:
```python
import numpy as np

# Problem data
```

```
unit_costs = np.array([14.0, 9.5, 22.0, 11.0, 17.5])
quantities = np.array([30, 45, 10, 60, 25])

# (a) Cost per route


# (b) Total network cost


# (c) Most expensive route
```

1.5.2 Exercise 2 — Route Classifier

Write a function `classify_routes(costs)` that takes a NumPy array of route costs and returns a list of labels (`"cheap"`, `"moderate"`, `"expensive"`) using the thresholds from the Control Flow section (< 500 cheap, < 1000 moderate, else expensive). Test it on `np.array([420, 850, 1500, 300, 990])`.

[18]:
```
def classify_routes(costs):
    labels = []
    # loop over costs and append the correct label to `labels`

    return labels

test_costs = np.array([420, 850, 1500, 300, 990])
# call classify_routes on test_costs and print the result
```

1.5.3 Exercise 3 — Budget Check

A logistics manager has a budget of 50,000 to ship five packages at the unit costs and quantities given below. Write a function `within_budget(quantities, unit_costs, budget)` that returns a tuple `(is_within_budget, remaining_budget)`, then call it on the data below.

[19]:
```
import numpy as np

# Problem data
unit_costs = np.array([120.0, 340.0, 95.0, 210.0, 60.0])
quantities = np.array([50, 20, 80, 30, 100])
budget = 50000.0

def within_budget(quantities, unit_costs, budget):
    # compute total cost, then return (is_within_budget, remaining_budget)
    pass

# call within_budget and print the result
```

1.5.4 Exercise 4 — Distance Lookup

The distance matrix below (km) covers four cities, with a dictionary mapping each city name to its row/column index. Write code that looks up and prints the distance between `city_a` and `city_b` using the dictionary and 2D indexing — do not hardcode row/column numbers.

	Chennai	Bengaluru	Coimbatore	Madurai
Chennai	0	346	505	462
Bengaluru	346	0	365	460
Coimbatore	505	365	0	215
Madurai	462	460	215	0

[20]:
```python
import numpy as np

# Problem data
cities = {"Chennai": 0, "Bengaluru": 1, "Coimbatore": 2, "Madurai": 3}
distance_matrix = np.array([
    [0, 346, 505, 462],
    [346, 0, 365, 460],
    [505, 365, 0, 215],
    [462, 460, 215, 0]
])

city_a, city_b = "Bengaluru", "Madurai"

# look up the row/column indices for city_a and city_b using `cities`

# use those indices to get the distance from distance_matrix

# print the result
```

1.5.5 Exercise 5 — Mode Split Summary

A freight operator ships the following tonnage by mode this month: Road = 1,250, Rail = 3,400, Air = 180, Sea = 950. Using NumPy, compute (a) total tonnage, (b) the percentage share of each mode (rounded to one decimal place), and (c) the dominant mode.

[21]:
```python
import numpy as np

# Problem data
modes = ["Road", "Rail", "Air", "Sea"]
tonnage = np.array([1250, 3400, 180, 950])

# (a) Total tonnage


# (b) Percentage share per mode
```

```
# (c) Dominant mode
```

1.6 Further Reading

- *Python Software Foundation*, "The Python Tutorial," https://docs.python.org/3/tutorial/ — the official, authoritative introduction to the language.
- *NumPy Developers*, "NumPy Quickstart," https://numpy.org/doc/stable/user/quickstart.html — the standard entry point for array programming.
- *Sweigart, A.*, "Automate the Boring Stuff with Python," https://automatetheboringstuff.com/ — a free, beginner-friendly, project-based book.
- *VanderPlas, J.*, "A Whirlwind Tour of Python," https://jakevdp.github.io/WhirlwindTourOfPython/ — a fast, example-driven tour aimed at readers with some programming background.